# Guided Exercise 2b

**Adrianna Alvarez & Ava DeCristofaro**
Group 87: Team 9

# Syntactic Analysis: Definition of Test Cases

## Valid Input Cases

These test cases verify correct behavior when the input JSON fully complies with the defined grammar and value constraints.

- **TC1**: Valid JSON with `.pdf` extension
- **TC2**: Valid JSON with `.docx` extension
- **TC3**: Valid JSON with `.xlsx` extension

These cases ensure that:

- The JSON structure is correctly formed
- The `PROJECT_ID` is a valid 32-character hexadecimal string
- The `FILENAME` follows the required format (8 alphanumeric characters + valid extension)
- Full coverage of all valid `<EXTENSION>` branches is achieved.

## Invalid JSON Formatting

These test cases raise an exception when the input JSON content is syntactically incorrect and cannot be parsed:

- **TC4–TC5, TC7, TC9–TC10**: Missing brackets or incorrect JSON boundaries
- **TC13, TC15–TC17, TC19–TC20, TC22–TC24, TC27**: Missing or incorrect quotation marks
- **TC16, TC23, TC38–TC39, TC54–TC55**: Missing or incorrect colons
- **TC28–TC31**: Invalid JSON start/end symbols
- **TC32–TC37, TC40–TC41, TC44–TC45, TC48–TC49, TC52–TC53, TC56–TC57, TC66–TC67**: Duplicate or malformed quotation marks
- **TC46–TC47**: Incorrect field separators

These tests ensure that any malformed JSON syntax is correctly detected and raise the exception “The file is not JSON formatted.”

## Invalid JSON Structure

These test cases catch an exception when the input JSON content is syntactically valid but does not conform to the expected structure:

- **TC6**: Missing required fields (`PROJECT_ID`, `FILENAME`)
- **TC11–TC12**: Duplicate fields (`PROJECT_ID`, `FILENAME`)
- **TC14, TC21**: Invalid or missing field labels
- **TC34–TC35, TC50–TC51**: Modified or duplicated field labels resulting in invalid keys

These tests ensure that the JSON structure strictly follows the defined grammar. If not, the method should raise the exception "JSON does not have the expected structure."

## JSON Data Has No Valid Values

These test cases raise an exception when the input JSON structure is correct, but the field values violate constraints from the grammar:

- **TC18, TC42–TC43**: Invalid `PROJECT_ID` values (not valid MD5 format)
- **TC25, TC58–TC59**: Invalid file name (not exactly 8 alphanumeric characters)
- **TC26, TC60–TC62**: Missing or duplicated file extensions
- **TC63–TC65**: Unsupported file extensions (e.g., `.ppt`)

These tests ensure that structurally valid JSON inputs are still validated against strict value constraints. Otherwise, the method should raise the exception "JSON data has no valid values"

## Input File Does Not Exist

This test covers the case where the provided JSON file path does not correspond to an existing file:

- **TC68**: File path does not exist

This ensures that the system properly raises the "Input file not found." exception before attempting to process the content.

## Internal Processing Error

This test simulates a failure during internal processing, specifically during the generation of the file signature in order to trigger the exception “Internal processing error when getting the file_signature.”:

- **TC69**: Valid JSON input with forced failure during signature generation

This test ensures that if an unexpected runtime error occurs, then the system correctly handles it by raising an exception.

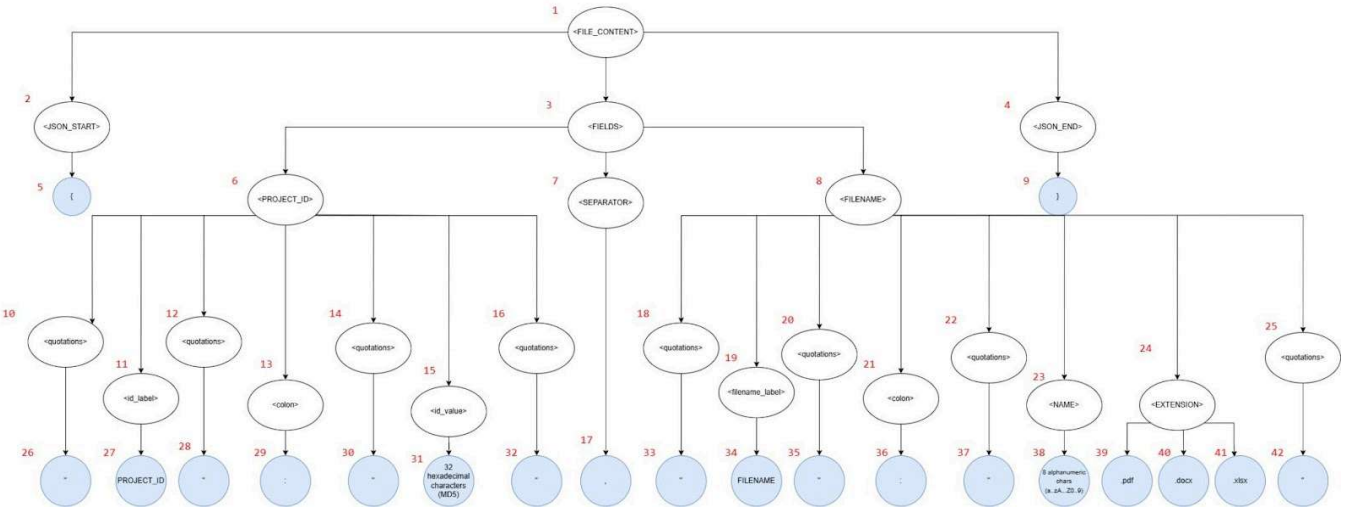
---

# Grammar:

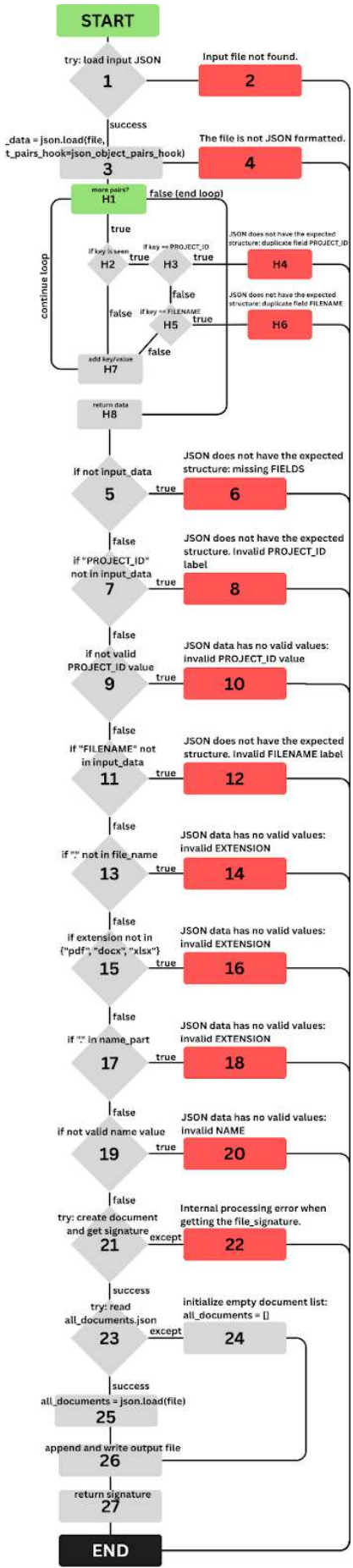
- <FILE_CONTENT> := <json_start> <FIELDS> <json_end>
- <FIELDS> := <PROJECT_ID> <separator> <FILENAME>
- <PROJECT_ID> := <quotations> <id_label> <quotations> <colon> <quotations> <id_value> <quotations>
- <FILENAME> := <quotations> <name_label> <quotations> <colon> <quotations> <name> <extension> <quotations>

- <id_label> := PROJECT_ID
- <id_value> := 32 hexadecimal characters (MD5)
- <filename_label> := FILENAME
- <name> := 8 alphanumeric chars
- <extension> := .pdf | .docx | .xlsx
- <separator> := ,
- <colon> := :
- <quotations> := "
- <json_start> := {
- <json_end> := }

# Derivation Tree:



# Structural Test Technique



Complexity: 47E - 33N + 2 = 16 paths